

Exercise on Variational Autoencoders with PyTorch

Goal: Build a Variational Autoencoder that reconstructs images of rectangles and circles.

Let's start by importing the necessary modules. We are going to work with the `pytorch` library.

```
In [1]: import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from torch.utils.data import Dataset, DataLoader
```

Here we will build our dataset of images using `numpy` arrays and then transforming them to `pytorch` tensors.

```
In [2]: #####
# Parameters
#####
N = 4000      # shapes per class
filled = False # filled shapes or boundary only
img_size = 16 # output image size

S = 400 if filled else 60 # points per shape

#####
# Function to convert points to centered image
#####
def shape_to_image(points, img_size=64):
    """
    Convert a shape (Nx2 points) into a centered grayscale image of size img_size x img_size.
    """
    # Center the shape
    min_xy = points.min(axis=0)
    max_xy = points.max(axis=0)
    center = (min_xy + max_xy) / 2
    points_centered = points - center

    # Scale to fit the image
    max_extent = np.max(max_xy - min_xy) / 2
    scale = (img_size * 0.4) / max_extent # Leave margin
    points_scaled = points_centered * scale

    # Shift to image coordinates
    img_coords = points_scaled + img_size / 2
    img_coords = np.round(img_coords).astype(int)

    # Rasterize points
    img = np.zeros((img_size, img_size))
    for x, y in img_coords:
        if 0 <= x < img_size and 0 <= y < img_size:
            img[y, x] = 1 # y=row, x=col
    return img

#####
# Generate Rectangles
#####
if not filled:
    xy = np.random.randint(0,2, size=S*N)
    side = np.random.randint(0,2, size=S*N)
    a = np.random.uniform(0,1, size=S*N)*xy + side*(1-xy)
    b = np.random.uniform(0,1, size=S*N)*(1-xy) + side*xy
else:
    a = np.random.uniform(0,1, size=S*N)
    b = np.random.uniform(0,1, size=S*N)

w = np.repeat(np.random.uniform(3,10, size=N), S)
h = np.repeat(np.random.uniform(3,10, size=N), S)
px = np.repeat(np.random.uniform(1,90, size=N), S)
py = np.repeat(np.random.uniform(1,90, size=N), S)

rx = (a*w + px).reshape(N,-1)
ry = (b*h + py).reshape(N,-1)
X_rect = np.stack([rx, ry], axis=2)
print("Rectangle points:", X_rect.shape)

#####
# Generate Circles
#####
```

```

theta = np.random.uniform(0, 2*np.pi, S*N)
radius = (np.random.uniform(0,1, S*N)**0.5) if filled else np.ones(S*N)

scale = np.repeat(np.random.uniform(3,10, size=N), S)
ox = np.repeat(np.random.uniform(10,90, size=N), S)
oy = np.repeat(np.random.uniform(10,90, size=N), S)

cx = (radius * np.cos(theta) * scale + ox).reshape(N,-1)
cy = (radius * np.sin(theta) * scale + oy).reshape(N,-1)
X_circ = np.stack([cx, cy], axis=2)
print("Circle points:", X_circ.shape)

#####
# Convert all shapes to images
#####
X_images = []
y_labels = []

for shape, label in zip([X_rect, X_circ], [0, 1]):
    for i in range(N):
        img = shape_to_image(shape[i], img_size)
        X_images.append(img)
        y_labels.append(label)

X_images = np.array(X_images) # shape: (2*N, img_size, img_size)
y_labels = np.array(y_labels).reshape(-1, 1)
print("Image dataset:", X_images.shape)

#####
# Split into train/test sets
#####
X_train, X_test, Y_train, Y_test = train_test_split(X_images, y_labels, test_size=0.5, random_state=1)

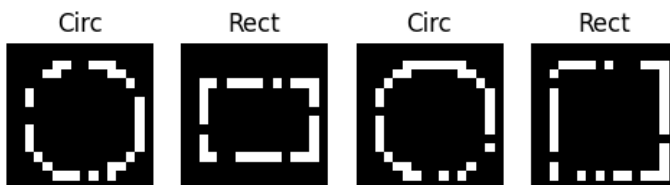
# Convert to PyTorch tensors
X_train = torch.tensor(X_train, dtype=torch.float32).unsqueeze(1) # add channel dim
X_test = torch.tensor(X_test, dtype=torch.float32).unsqueeze(1)
Y_train = torch.tensor(Y_train, dtype=torch.long)
Y_test = torch.tensor(Y_test, dtype=torch.long)

print("Train:", X_train.shape, "Test:", X_test.shape)

#####
# Visualize a few examples
#####
plt.figure(figsize=(6,3))
for i in range(4):
    plt.subplot(2,4,i+1)
    plt.imshow(X_train[i,0], cmap='gray')
    plt.title("Rect" if Y_train[i]==0 else "Circ")
    plt.axis('off')
plt.show()

```

Rectangle points: (4000, 60, 2)
 Circle points: (4000, 60, 2)
 Image dataset: (8000, 16, 16)
 Train: torch.Size([4000, 1, 16, 16]) Test: torch.Size([4000, 1, 16, 16])



This cell prepares our generated data to be used in a PyTorch training loop: `ShapeDataset` is a custom PyTorch `Dataset` class that wraps the image tensors (`X`) and their labels (`y`), allowing the data to be indexed and accessed efficiently. `DataLoader` wraps the training and testing datasets to automatically handle creating mini-batches (`batch_size=32`) and shuffling the data (for the training set) during the model's training and evaluation phases.

```

In [3]: class ShapeDataset(Dataset):
        def __init__(self, X, y=None):
            self.X = X
            self.y = y

        def __len__(self):
            return len(self.X)

        def __getitem__(self, idx):
            if self.y is not None:
                return self.X[idx], self.y[idx]
            return self.X[idx]

```

```
batch_size=32
train_loader = DataLoader(ShapeDataset(X_train, Y_train), batch_size=batch_size, shuffle=True)
test_loader = DataLoader(ShapeDataset(X_test, Y_test), batch_size=batch_size)
```

Here you will find the actual VAE which we will be training. It is a class (FlatImageVAE) of type `nn.Module` which works like this:

- The images are flattened before being fed to the encoder.
- The encoder is just a simple MLP written using `torch.nn`. The output of the encoder will be two vectors, one containing the mean values of the gaussian distributions and one containing the logarithm of the variance values. These vectors will have the same dimensionality of the latent space (20 by default).
- In between the encoder and the decoder the `reparameterize()` function implements the reparametrization trick.
- The decoder is another MLP that will reconstruct the image starting from the encoder output.
- The `forward()` function describes how the operations above are carried out i.e. lets the VAE perform the forward pass. You can find more information about it here: <https://www.codegenes.net/blog/netmodule-pytorch/>

```
In [4]: #####
# VAE for flattened images
#####
class FlatImageVAE(nn.Module):
    def __init__(self, img_size=10, hidden_dim=64, latent_dim=20):
        super().__init__()
        self.img_size = img_size
        self.in_dim = img_size * img_size # flatten image

        # Encoder
        self.encoder = nn.Sequential(
            nn.Linear(self.in_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU()
        )
        self.mu = nn.Linear(hidden_dim, latent_dim)
        self.logvar = nn.Linear(hidden_dim, latent_dim)

        # Decoder
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, self.in_dim),
            nn.Sigmoid() # output in [0,1]
        )

    def encode(self, x):
        h = self.encoder(x)
        return self.mu(h), self.logvar(h)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5*logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z):
        out = self.decoder(z)
        return out.view(-1, 1, self.img_size, self.img_size)

    def forward(self, x):
        x = x.view(x.size(0), -1) # flatten
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        recon = self.decode(z)
        return recon, mu, logvar
```

The `vae_loss()` function implements the negative ELBO derived in the last section. It computes the two distinct components of the loss:

- Reconstruction Loss (BCE): Assuming a Bernoulli distribution for the pixel values, the negative log-likelihood becomes the BCE loss. The code flattens both the reconstructed image and the target image to compute this loss pixel by pixel.
- KL Divergence (KLD): This applies the closed-form analytical formula. For numerical stability, the encoder outputs the log-variance ($\log \sigma^2$) rather than the variance (σ^2). By letting $\logvar = \log \sigma^2$ and $\exp(\logvar) = \sigma^2$, the theoretical formula is algebraically rearranged in the code as:

$$D_{KL} = -0.5 \sum (1 + \log \text{var} - \mu^2 - \exp(\log \text{var}))$$

Finally, the function returns the combined loss. Note that the reconstruction term is divided by 2 (`BCE/2.`): this is a common empirical weighting trick used to balance the strength of the reconstruction against the spatial regularization of the KL divergence.

```
In [5]: def vae_loss(recon, x, mu, logvar):
# Flatten for BCE
recon_flat = recon.view(recon.size(0), -1)
x_flat = x.view(x.size(0), -1)
BCE = nn.functional.binary_cross_entropy(recon_flat, x_flat, reduction='sum')
# KL divergence
KLD = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
#print("bce",BCE/2., "kld",KLD)
return BCE/2. + KLD
```

Now let's train the VAE. You can try to change the number of epochs or the latent space dimension to see what happens.

```
In [6]: device = "cuda" if torch.cuda.is_available() else "cpu"

model = FlatImageVAE(img_size=16, hidden_dim=348, latent_dim=2).to(device)
optimizer = optim.Adam(model.parameters(), lr=1e-4)
epochs = 50

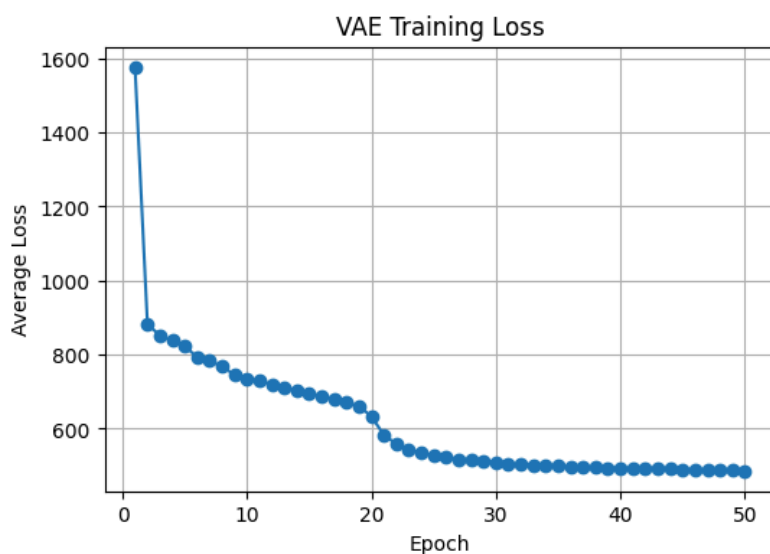
# List to store average loss per epoch
losses = []

for epoch in range(epochs):
    model.train()
    total_loss = 0
    for X_batch, y_batch in train_loader:
        X_batch = X_batch.to(device)
        optimizer.zero_grad()
        recon, mu, logvar = model(X_batch)
        loss = vae_loss(recon, X_batch, mu, logvar)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    avg_loss = total_loss / len(train_loader)
    losses.append(avg_loss)
    #print(f"Epoch {epoch+1}/{epochs}, Loss: {avg_loss:.4f}")

# Plot Loss vs Epoch
import matplotlib.pyplot as plt

plt.figure(figsize=(6,4))
plt.plot(range(1, epochs+1), losses, marker='o')
plt.xlabel("Epoch")
plt.ylabel("Average Loss")
plt.title("VAE Training Loss")
plt.grid(True)
plt.show()
```



Now let's try the VAE on the test dataset. Notice that to do so the `model` must be set to evaluation mode using the `.eval()` method. You can find everything you need to know about it here: <https://www.codegenes.net/blog/nnmoduleeval-in-pytorch/>

```
In [7]: model.eval() # set model to evaluation mode
```

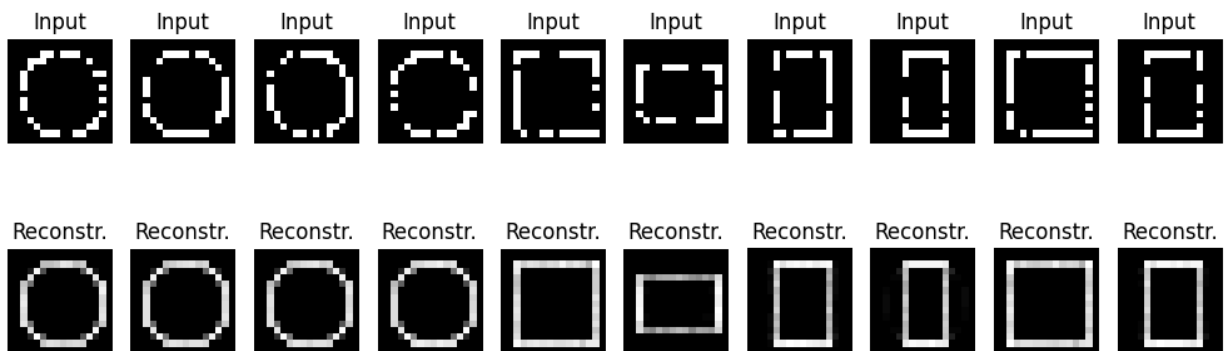
```
# Get a batch from test_loader
X_batch, Y_batch = next(iter(test_loader))
X_batch = X_batch.to(device)

# Forward pass through VAE
with torch.no_grad():
    recon, mu, logvar = model(X_batch)

# Move to CPU and numpy for plotting
X_batch = X_batch.cpu().numpy()
recon = recon.cpu().numpy()
print(recon.shape)
Y_batch = Y_batch.cpu().numpy()

# Plot first 5 examples
num_examples = 10
plt.figure(figsize=(10,4))
for i in range(num_examples):
    # Input
    plt.subplot(2, num_examples, i+1)
    plt.imshow(X_batch[i,0], cmap='gray')
    plt.title("Input")
    plt.axis('off')
    # Reconstruction
    plt.subplot(2, num_examples, i+1+num_examples)
    plt.imshow(recon[i,0], cmap='gray')
    plt.title("Reconstr.")
    plt.axis('off')
plt.tight_layout()
plt.show()
```

(32, 1, 16, 16)



As you can see, it is not doing too badly, considering that you are compressing the input information into a very low-dimensional latent space. What happens if you increase the latent space dimensionality?

This cell evaluates the trained VAE on the test dataset to visualize how the model organizes the different shape classes within the learned latent space.

- `model.eval()` ensures the model is in evaluation mode just like before, while `torch.no_grad()` ensures that gradients are not tracked, saving memory and computation time during inference.
- The core logic inside the loop handles the probabilistic sampling. After the encoder outputs the mean (`mu`) and log-variance (`logvar`), the code samples the latent vector `z` from the distribution $q(z|x)$. It does this using the reparameterization trick:
 1. It converts the log-variance to standard deviation: `std = torch.exp(0.5 * logvar)`
 2. It samples random noise from a standard normal distribution: `eps = torch.randn_like(std)`
 3. It scales and shifts the noise to match the predicted distribution: `z = mu + eps * std`
- Finally, the code collects all the sampled `z` vectors and their corresponding true labels. It then creates a scatter plot of the first two dimensions of the latent space (`z1` vs `z2`), coloring the points by class to reveal whether the VAE has successfully learned to group similar shapes together while keeping the classes distinct.

```
In [10]: model.eval()
all_z = []
all_labels = []

with torch.no_grad():
    for X_batch, Y_batch in test_loader:
        X_batch = X_batch.to(device)
        mu, logvar = model.encode(X_batch.view(X_batch.size(0), -1))

        # Sample z from q(z|x)
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
```

```

z = mu + eps * std

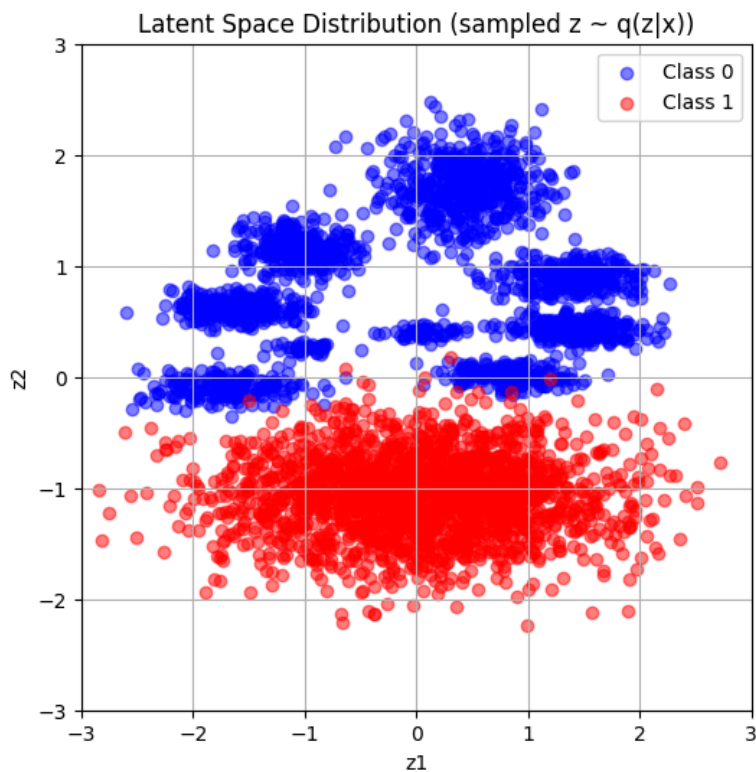
all_z.append(z.cpu().numpy())
all_labels.append(Y_batch.numpy())

all_z = np.concatenate(all_z, axis=0)
all_labels = np.concatenate(all_labels, axis=0)

# Plot first two dimensions
plt.figure(figsize=(6,6))
for label, color in zip([0,1], ['blue', 'red']):
    idxs = all_labels.flatten() == label
    plt.scatter(all_z[idxs,0], all_z[idxs,1], alpha=0.5, label=f'Class {label}', color=color)

plt.xlabel("z1")
plt.ylabel("z2")
plt.title("Latent Space Distribution (sampled  $z \sim q(z|x)$ ")
plt.legend()
plt.grid(True)
plt.xlim(-3,3) # adjust as needed
plt.ylim(-3,3)
plt.show()

```



As you can see the class 1 is well represented by a gaussian distribution with mean 0 and variance 1, but the training can still be optimized further for the class 0. However, for our purposes, this is more than enough.